InspectionService 快速集成指南

1. 服务地址

```
POST http://localhost:8088/inspect
```

2. 请求格式

```
{
  "content": "待审查的文本内容"
}
```

3. 响应格式

```
{
  "isSafe": true,      // true=安全, false=危险
  "reason": "Content is safe."
}
```

4. Java 集成示例

添加依赖（如果还没有）

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

创建客户端类

```
@Component
public class InspectionServiceClient {

    @Autowired
    private RestTemplate restTemplate;

    private static final String INSPECTION_URL = "http://localhost:8088/inspect";

    public boolean isContentSafe(String content) {
        try {
```

```
            HttpHeaders headers = new HttpHeaders();
            headers.setContentType(MediaType.APPLICATION_JSON);

            Map<String, String> request = Map.of("content", content);
            HttpEntity<Map<String, String>> entity = new HttpEntity<>(request,
headers);

            ResponseEntity<Map> response =
restTemplate.postForEntity(INSPECTION_URL, entity, Map.class);
            Map<String, Object> result = response.getBody();

            return (Boolean) result.get("isSafe");

        } catch (Exception e) {
            // 异常时默认认为安全，避免影响主业务
            return true;
        }
    }
}
```

配置 RestTemplate

```
@Configuration
public class RestTemplateConfig {
    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

5. 使用示例

在 Controller 中使用

```
@RestController
public class PostController {

    @Autowired
    private InspectionServiceClient inspectionClient;

    @PostMapping("/posts")
    public ResponseEntity<String> createPost(@RequestBody PostRequest request) {
        // 审查内容
        if (!inspectionClient.isContentSafe(request.getContent())) {
            return ResponseEntity.badRequest().body("内容包含危险信息，无法发布");
        }

        // 继续正常业务逻辑
        // ...
```

```
        return ResponseEntity.ok("发布成功");
    }
}
```

在 Service 中使用

```
@Service
public class CommentService {

    @Autowired
    private InspectionServiceClient inspectionClient;

    public void addComment(String content) {
        if (!inspectionClient.isContentSafe(content)) {
            throw new IllegalArgumentException("评论内容不符合规范");
        }

        // 保存评论
        // ...
    }
}
```

6. 最佳实践

异步调用（推荐）

```
@Async
public CompletableFuture<Boolean> validateContentAsync(String content) {
    return CompletableFuture.supplyAsync(() -> isContentSafe(content));
}
```

批量检查

```
public Map<String, Boolean> validateMultipleContent(List<String> contents) {
    return contents.stream()
        .collect(Collectors.toMap(
            content -> content,
            content -> isContentSafe(content)
        ));
}
```

7. 错误处理

- **网络异常**: 默认返回 `true`（安全）
- **服务不可用**: 默认返回 `true`（安全）

- **超时**: 建议设置10秒超时

8. 测试命令

```
# 测试危险内容
curl -X POST http://localhost:8088/inspect \
  -H "Content-Type: application/json" \
  -d '{"content": "我想结束生命"}'

# 预期响应
# {"isSafe": false, "reason": "Content is not safe."}
```